

DeepXube Domain Design Document

Supply Chain Logistics: Multi-Commodity Warehouse Routing

J.C. Vaught

April 2026

1 Overview

This document specifies a *Supply Chain Logistics* domain for the DeepXube framework. The problem models routing discrete packages through a fixed network of warehouses with per-node capacity constraints. Each warehouse holds integer quantities of multiple commodity types, and the solver must redistribute packages from an arbitrary initial configuration to a target distribution using single-unit shipments along network edges. The domain implements the `Domain` abstract base class hierarchy including `ActsEnum`, `GoalStartRevWalkableActsRev`, `HasFlatSGIn`, `StateGoalVizable`, and `StringToAct`.

2 State Representation

A state encodes the inventory of every commodity at every warehouse. For a network with W warehouses and K commodity types, the state is an integer matrix $\mathbf{S} \in \mathbb{Z}_{\geq 0}^{W \times K}$ where $S_{w,k}$ is the number of units of commodity k stored at warehouse w . This matrix is flattened into a one-dimensional `numpy` array of shape $(W \cdot K,)$ with dtype `int8`.

The default configuration uses $W = 6$ warehouses and $K = 4$ commodity types, yielding a state vector of length 24. Each warehouse has a maximum total capacity C_w (summed over all commodities). The default capacity is $C_w = 8$ for all warehouses. The total number of units in the system is conserved. A reasonable default is 20 total units distributed across the 4 commodity types (5 units each).

```
State class: SCState
inventory : NDArray[np.int8], shape (W · K,)
__hash__ : hash(self.inventory.tobytes())
```

The `int8` dtype supports values in $[0, 127]$, which is more than sufficient for per-warehouse-per-commodity counts. For the default parameters, individual cell values range from 0 to 8.

3 Goal Representation

A goal specifies the target inventory distribution, represented identically to the state as a flattened array of shape $(W \cdot K,)$ with dtype `int8`. The goal array $\mathbf{G}$ satisfies the same conservation constraint: $\sum_{w,k} G_{w,k} = \sum_{w,k} S_{w,k}$. Additionally, every goal must satisfy per-warehouse capacity constraints, so $\sum_k G_{w,k} \leq C_w$ for all w .

```

Goal class: SCGoal
    target : NDArray[np.int8],  shape (W · K,)

```

Because the goal fully specifies all inventory levels (not a partial specification), the `is_solved` check and goal sampling are straightforward. This mirrors the NPuzzle domain, where the goal tile array has the same shape as the state tile array.

4 Network Topology

The warehouse network is a fixed undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $|\mathcal{V}| = W$ nodes. The default topology is a cycle with two cross-edges, giving each node degree 3 or 4 and $|\mathcal{E}| = 8$ undirected edges (16 directed). This topology is stored as an adjacency list computed at initialization. The graph must be connected to guarantee reachability.

Edges are bidirectional. Each directed edge $(u, v) \in \mathcal{E}$ allows shipping one unit of any commodity from warehouse u to warehouse v .

5 Actions

An action ships one unit of one commodity type along one directed edge. Each action is a tuple (e, k) where $e = (u, v)$ is a directed edge and $k \in \{0, \dots, K - 1\}$ is the commodity type. The total number of *nominal* actions is $|\mathcal{E}_{\text{directed}}| \times K$. For the default configuration this is $16 \times 4 = 64$ actions.

However, actions are *state-dependent*. An action (u, v, k) is valid only if both conditions hold.

$$S_{u,k} > 0 \quad (\text{source has stock}) \tag{1}$$

$$\sum_{k'} S_{v,k'} < C_v \quad (\text{destination has capacity}) \tag{2}$$

This makes the domain an `ActsEnum` rather than `ActsEnumFixed`. The `get_state_actions` method enumerates valid actions per state, analogous to how Hanoi filters moves that violate the disk-ordering constraint.

```

Action class: SCAction
    action_id : int  (encodes u, v, k)
    __hash__ : self.action_id

```

The action integer is computed as `edge_idx × K + k`, where `edge_idx` indexes into the sorted directed edge list.

6 Transition Dynamics

Applying action (u, v, k) to state $\mathbf{S}$ produces state $\mathbf{S}'$ by decrementing $S_{u,k}$ and incrementing $S_{v,k}$, leaving all other entries unchanged. The transition cost is 1.0 (uniform cost). If the action is invalid

(empty source or full destination), the state is unchanged and the transition cost is 1.0 as a no-op penalty, matching the NPuzzle convention for wall-bounce moves.

The key property that makes this domain *non-decomposable* is the shared capacity constraint. Even though commodities are distinguishable, they compete for warehouse space. Routing commodity k_1 through warehouse w consumes capacity that commodity k_2 may need. This creates a multi-commodity flow coupling. You cannot optimally route each commodity independently and combine the results, because an intermediate warehouse may need to temporarily hold units of commodity A to make room for commodity B to pass through. This is analogous to how Sokoban boxes block each other despite being independent targets.

7 Solved Check

The `is_solved` method performs an elementwise comparison between the state inventory array and the goal target array.

$$\text{is_solved}(\mathbf{S}, \mathbf{G}) = \bigwedge_{i=0}^{W \cdot K - 1} (S_i = G_i) \quad (3)$$

In NumPy, this is `np.all(states_np == goals_np, axis=1)` over a batch, identical to the Hanoi implementation.

8 Reversibility and Training Data Generation

Actions are naturally reversible. The reverse of shipping commodity k from u to v is shipping commodity k from v to u . The domain inherits from `ActsRev` and `GoalStartRevWalkableActsRev`. The `rev_action` method swaps the source and destination of the edge while preserving the commodity index.

The `sample_goalstate_goal_pairs` method generates valid (state, goal) pairs by constructing a random feasible inventory distribution. The procedure is as follows. First, for each commodity type, randomly distribute its total units across warehouses. Second, verify and adjust to ensure no warehouse exceeds its capacity. Third, the generated distribution serves as both the goal state and the goal.

The `random_walk_rev` method (inherited from `GoalStartRevWalkableActsRev`) performs a forward random walk from goal states, since forward and reverse walks are equivalent for this domain. At each step, a valid action is sampled uniformly from `get_state_actions`. The resulting scrambled state becomes the start state for training. The number of random walk steps controls problem difficulty.

9 Neural Network Input

The domain implements `HasFlatSGIn`. The `get_input_info_flat_sg` method returns `([W * K], [1])`, meaning the input is a flat vector of length $W \cdot K = 24$ with no one-hot encoding (depth 1). The `to_np_flat_sg` method stacks state inventory arrays into a batch. The goal is encoded in the same array by concatenation, giving a total input dimension of $2 \times W \times K = 48$. However, since the goal has the same shape as the state, it may be more effective to return a single array of shape $(W \cdot K,)$ and pass both state and goal as separate inputs, allowing the `resnet_fc` heuristic to process the concatenated $(2 \times W \times K,)$ input.

10 Visualization

The visualization renders the warehouse network as a graph using `matplotlib`. Warehouses are drawn as rounded rectangular nodes arranged in a circular or force-directed layout. Each node is subdivided into K horizontal bands, where the fill level of each band represents the current inventory of that commodity as a fraction of the per-commodity capacity implied by C_w . Edges are drawn as curves connecting warehouses, with arrowheads indicating shipping directions.

The color scheme uses the brand palette. Each commodity type receives a distinct accent color: commodity 0 uses Garnet (`#73000A`), commodity 1 uses Atlantic (`#466A9F`), commodity 2 uses Horseshoe (`#65780B`), and commodity 3 uses Honeycomb (`#A49137`). Node outlines use Congaree (`#1F414D`). The background is Black_10 (`#ECECEC`). Edges carrying active shipments are highlighted with Rose (`#CC2E40`) with a glowing effect.

For GIF animation, each frame shows one action. When a shipment occurs, a small colored dot (matching the commodity color) animates along the edge from source to destination. The source node’s fill level decreases and the destination’s increases. Warehouse nodes that match their goal level pulse with a subtle green (Horseshoe) border. Warehouses at full capacity display a small warning indicator. The goal distribution is shown as a ghost overlay with dashed outlines on each node. The animation interpolates between states over 4-5 subframes per action for smooth motion, targeting 10 fps in the output GIF.

11 Recommended Configurations and State Space

The state space size for W warehouses, K commodities, $C_w = C$ uniform capacity, and N_k total units of commodity k is the product of multinomial coefficients constrained by capacity. For the default $W = 6$, $K = 4$, $C = 8$, $N_k = 5$, the state space is approximately 10^9 to 10^{11} states, which is comparable to the 15-puzzle ($\approx 10^{13}$) and well within the range where DeepXube’s learned heuristic approach is necessary since BFS is infeasible.

Smaller configurations for testing and BFS verification include the following.

Config	W	K	C	Total units	Approx. states
Tiny	3	2	4	6	$\sim 10^3$
Small	4	3	6	12	$\sim 10^5$
Medium	6	4	8	20	$\sim 10^{10}$
Large	8	4	10	32	$\sim 10^{14}$

Table 1: Suggested configurations with approximate state space sizes.

The Tiny and Small configurations are suitable for BFS-based optimal solution verification during development. The Medium configuration is the default training target. The Large configuration serves as a stretch goal.

12 Challenges and Tradeoffs

Capacity-induced dead ends. Unlike the Rubik’s Cube or NPuzzle, capacity constraints can create states from which certain actions are blocked. A warehouse at full capacity cannot receive shipments. If the network has bottleneck edges (bridges), congestion can force long detours. The random walk procedure naturally avoids truly dead states because every action that adds to a

warehouse also removes from another, preserving total inventory. However, near-dead-end states (where most actions are blocked) can arise and slow search significantly.

Multi-commodity flow complexity. The theoretical optimal solution corresponds to solving an integer multi-commodity flow problem, which is NP-hard in general. The capacity coupling between commodity types means the heuristic function must learn to account for contention. A decomposed heuristic (routing each commodity independently, ignoring capacity) would be inadmissible because it underestimates the true cost.

Action space size. With 64 nominal actions (only a subset valid per state), the branching factor is moderate. For the Large configuration ($W = 8$, 20 directed edges, $K = 4$), the nominal action count rises to 80. This is manageable for A* with a good heuristic but expensive for uninformed search.

Heuristic decomposition. A useful lower bound is the sum over all commodities of the minimum transport cost for that commodity type, ignoring capacity. This can be precomputed as shortest-path distances between all warehouse pairs. The learned heuristic should improve upon this lower bound by capturing capacity interactions.

Random walk length calibration. Short random walks (< 10 steps) produce trivial instances where only a few units are displaced. Long walks (> 100 steps) may produce states that appear random and are hard to correlate with walk length. A range of 5 to 50 steps, sampled uniformly, provides good curriculum learning. The domain parser should accept the network size as `supply_chain.W.K.C`, for example `supply_chain.6.4.8`.